МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного образовательного
учреждения высшего
образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Сопроводительная записка по дисциплине

«Поддержка и тестирование программных модулей»

по теме:

«Создание и тестирование калькулятора»

Руководитель

_____/Берестов Д.Б./

«___» _____2026

Исполнитель

_____/Ворошнин И.О./

«___» _____2026 г.

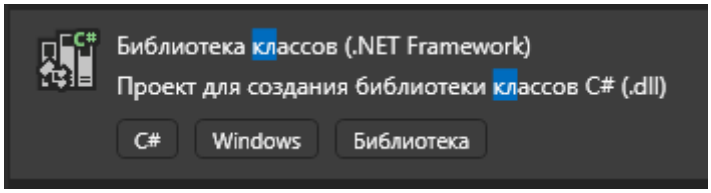
Пермь 2026

Содержание

Создание проекта библиотеки классов 3
Метод Execute 3
Проект модульных тестов 4
Написание теста для сложения 4
Тестирование остальных операций 5
Тест на деление на ноль 6
Метод Run 7
Тестирование метода Run 8
Создание WPF-приложения 9
Проектирование интерфейса WPF 10
Обработка нажатий кнопок 11
Запуск и финальное тестирование 11
Вывод 12

Создание проекта библиотеки классов

Создал проект из шаблона «Библиотека классов (.NET Framework)» (Рис. 1)



(Рис. 1)

Переименовал class1.cs в Calculator.cs

Метод Execute

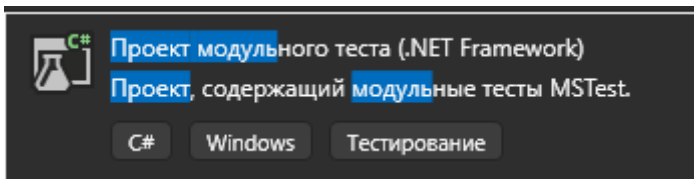
В классе Calculator создал метод Execute, который принимает два числа и символ операции. Через конструкцию switch реализовал сложение, вычитание, умножение, деление и возведение в степень. Для деления добавил проверку: если делитель равен нулю, выбрасывается исключение DivideByZeroException. После написания кода собрал проект — ошибок компиляции нет (Рис. 2)

```
public static class Calculator
{
    Ссылок: 0
    public static double Execute(double first, char op, double second)
    {
        switch (op)
        {
            case '+': return first + second;
            case '-': return first - second;
            case '*': return first * second;
            case '/':
                if (second == 0)
                    throw new DivideByZeroException("Деление на ноль");
                return first / second;
            case '^': return Math.Pow(first, second);
            default: throw new ArgumentException("Неизвестный оператор");
        }
    }
```

(Рис. 2)

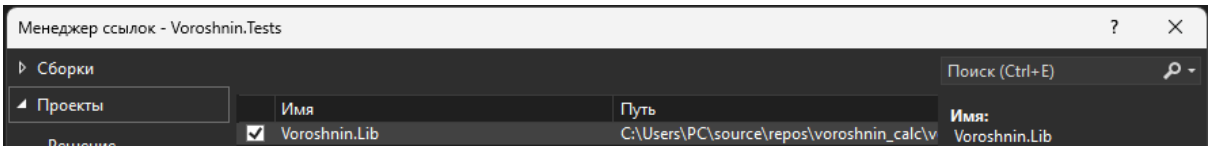
Проект модульных тестов

Для автоматической проверки логики добавил в решение проект типа «Проект модульного теста» с именем Voroshnin.Tests. (Рис. 3)



(Рис. 3)

Стандартный файл UnitTest1.cs переименовал в CalculatorTests.cs. Чтобы тесты видели классы библиотеки, добавил ссылку на проект Voroshnin.Lib. (Рис. 4)



(Рис. 4)

Сборка решения прошла успешно.

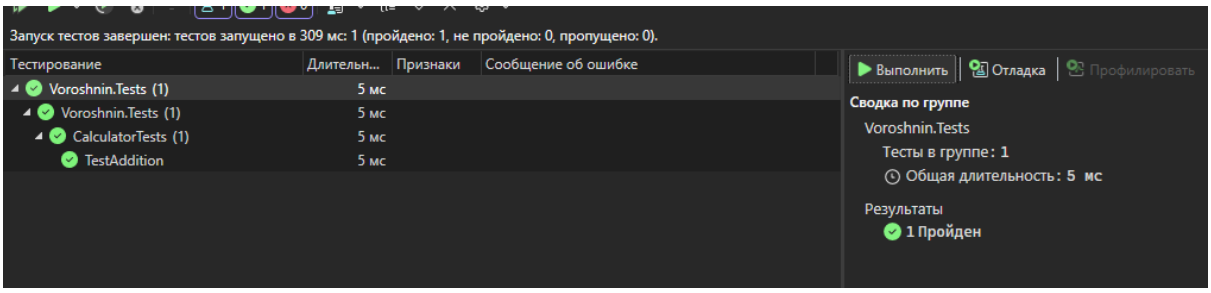
Написание теста для сложения

В классе CalculatorTests создал первый тестовый метод TestAddition с атрибутом [TestMethod]. В методе вызвал Calculator.Execute(5, '+', 3) и проверил результат через Assert.AreEqual(8, result). (Рис. 5)

```
namespace Voroshnin.Tests
{
    [TestClass]
    Ссылок: 0
    public class CalculatorTests
    {
        [TestMethod]
        Ссылок: 0
        public void TestAddition()
        {
            double result = Calculator.Execute(5, '+', 3);
            Assert.AreEqual(8, result);
        }
```

(Рис. 5)

После сборки запустил тесты через Обозреватель тестов — тест прошёл успешно. (Рис. 6)



(Рис. 6)

Тестирование остальных операций

После успешного теста на сложение добавил методы для проверки вычитания (TestSubtraction), умножения (TestMultiplication), деления (TestDivision) и возведения в степень (TestPower). В каждом тесте вызывал метод Execute с конкретными параметрами и проверял результат через Assert.AreEqual (Рис. 7)

```
[TestMethod]
Ссылок: 0
public void TestSubtraction()
{
    double result = Calculator.Execute(10, '-', 4);
    Assert.AreEqual(6, result);
}

[TestMethod]
Ссылок: 0
public void TestMultiplication()
{
    double result = Calculator.Execute(7, '*', 6);
    Assert.AreEqual(42, result);
}

[TestMethod]
Ссылок: 0
public void TestDivision()
{
    double result = Calculator.Execute(20, '/', 4);
    Assert.AreEqual(5, result);
}

[TestMethod]
Ссылок: 0
public void TestPower()
{
    double result = Calculator.Execute(2, '^', 3);
    Assert.AreEqual(8, result);
```

(Рис. 7)

Запустил все тесты — все 5 тестов прошли успешно (Рис. 8)

Тестирование	Длительн...	Признаки	Сообщение об ошибке
Voroshnin.Tests (5)	5 мс		
Voroshnin.Tests (5)	5 мс		
CalculatorTests (5)	5 мс		
TestAddition	5 мс		
TestDivision	< 1 мс		
TestMultiplication	< 1 мс		
TestPower	< 1 мс		
TestSubtraction	< 1 мс		

Выполнить | Отладка | Профилировать

Сводка по группе

Voroshnin.Tests

Тесты в группе: 5

Общая длительность: 5 мс

Результаты

5 Пройден

(Рис. 8)

Тест на деление на ноль

Для проверки корректной обработки ошибок добавил метод TestDivisionByZero атрибутом [ExpectedException (typeof (DivideByZero Exception))]. Этот атрибут указывает, что тест считается пройденным,

только если метод выдаст исключение DivideByZeroException. В теле теста вызвал Calculator.Execute(10, '/', 0). (Рис. 9)

```
}
[TestMethod]
[ExpectedException(typeof(DivideByZeroException))]
Ссылок: 0
public void TestDivisionByZero()
{
    Calculator.Execute(10, '/', 0);
}
```

(Рис. 9)

Тест прошёл успешно, что подтверждает правильную обработку деления на ноль в методе Execute. (Рис. 10)

Voroshnin.Tests (6)	6 мс
Voroshnin.Tests (6)	6 мс
CalculatorTests (6)	6 мс
TestAddition	5 мс
TestDivision	< 1 мс
TestDivisionByZero	1 мс
TestMultiplication	< 1 мс
TestPower	< 1 мс
TestSubtraction	< 1 мс

(Рис. 10)

Метод Run

Добавил в класс Calculator метод Run, который принимает строковое выражение и возвращает результат вычисления. Метод использует алгоритм сортировочной станции: два стека (для чисел и операторов), учёт приоритетов операций и обработку скобок. Также добавил вспомогательные методы: IsStartOfNegative (определение отрицательного числа), IsOperator (проверка оператора), GetPriority (приоритет операции) и ProcessOperator (выполнение операции).

В начале файла добавил using System.Globalization;

После добавления кода собрал проект — ошибок нет (Рис. 10)

```
Ссылок: 0
public static double Run(string expression)
{
    if (string.IsNullOrWhiteSpace(expression))
        throw new ArgumentException("Пустое выражение");

    expression = expression.Replace(",", ".").Replace(" ", "");

    var numbers = new Stack<double>();
    var operators = new Stack<char>();

    int i = 0;
    while (i < expression.Length)
    {
        char c = expression[i];

        if (char.IsDigit(c) || c == '.' || (c == '-' && IsStartOfNegative(expression, i)))
        {
            string numStr = "";
            if (c == '-') { numStr += c; i++; }

            while (i < expression.Length && (char.IsDigit(expression[i]) || expression[i] == '.'))
            {
                numStr += expression[i++];
            }

            if (double.TryParse(numStr, System.Globalization.NumberStyles.Any, System.Globalization.CultureInfo.InvariantCulture, out double num))
                numbers.Push(num);
            else
                throw new FormatException("Неверное число: " + numStr);
            continue;
        }
        else if (c == '(')
        {
            operators.Push(c);
            i++;
        }
        else if (c == ')')
        {
            while (operators.Count > 0 && operators.Peek() != '(')
                ProcessOperator(numbers, operators);

            if (operators.Count == 0)
                throw new FormatException("Нет открывающей скобки");

            operators.Pop();
            i++;
        }
        else if (IsOperator(c))
        {
            while (operators.Count > 0 &&
                   operators.Peek() != '(' &&
                   GetPriority(operators.Peek()) >= GetPriority(c))
            {
                ProcessOperator(numbers, operators);
            }
            operators.Push(c);
            i++;
        }
        else
        {
            throw new FormatException("Неверный символ: " + c);
        }
    }
```

Проблемы не найдены.

(Рис. 10)

Тестирование метода Run

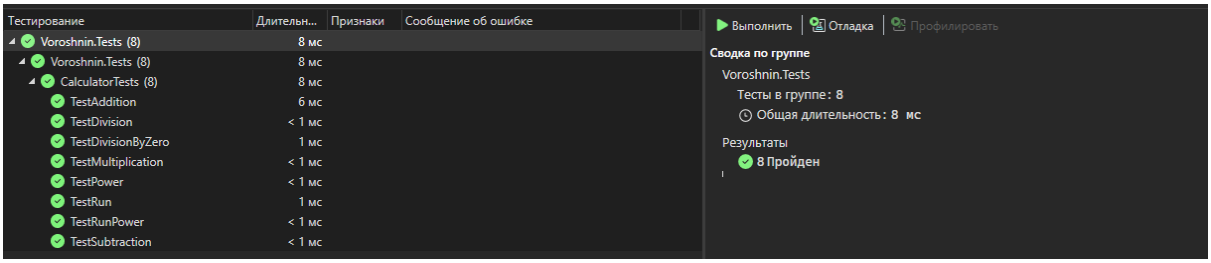
Для проверки метода Run создал два теста: TestRun (проверка простого выражения "2+3") и TestRunPower (проверка возведения в степень "2^4"). В тестах вызывал Calculator.Run со строковым выражением и проверял результат через Assert.AreEqual. (Рис. 11)

```
        [TestMethod]
        Ссылок: 0
        public void TestRun()
        {
            double result = Calculator.Run("2+3");
            Assert.AreEqual(5, result);
        }

        [TestMethod]
        Ссылок: 0
        public void TestRunPower()
        {
            double result = Calculator.Run("2^4");
            Assert.AreEqual(16, result);
        }
    }
```

(Рис. 11)

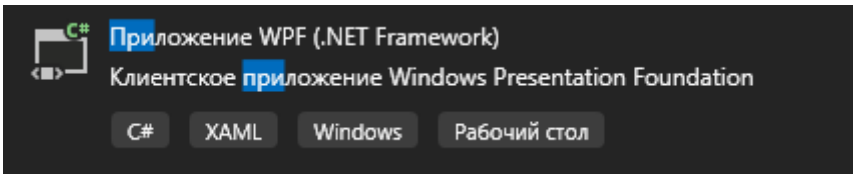
Запустил все тесты — все 8 тестов прошли успешно. (Рис. 12)



(Рис. 12)

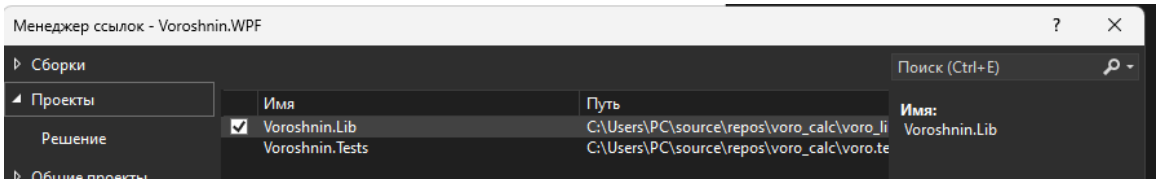
Создание WPF-приложения

Для интерфейса добавил в решение проект типа «Приложение WPF» с именем Voroshnin.WPF (Рис. 13)



(Рис. 13)

Чтобы интерфейс мог использовать логику калькулятора, добавил ссылку на проект Voroshnin.Lib (Рис. 14)



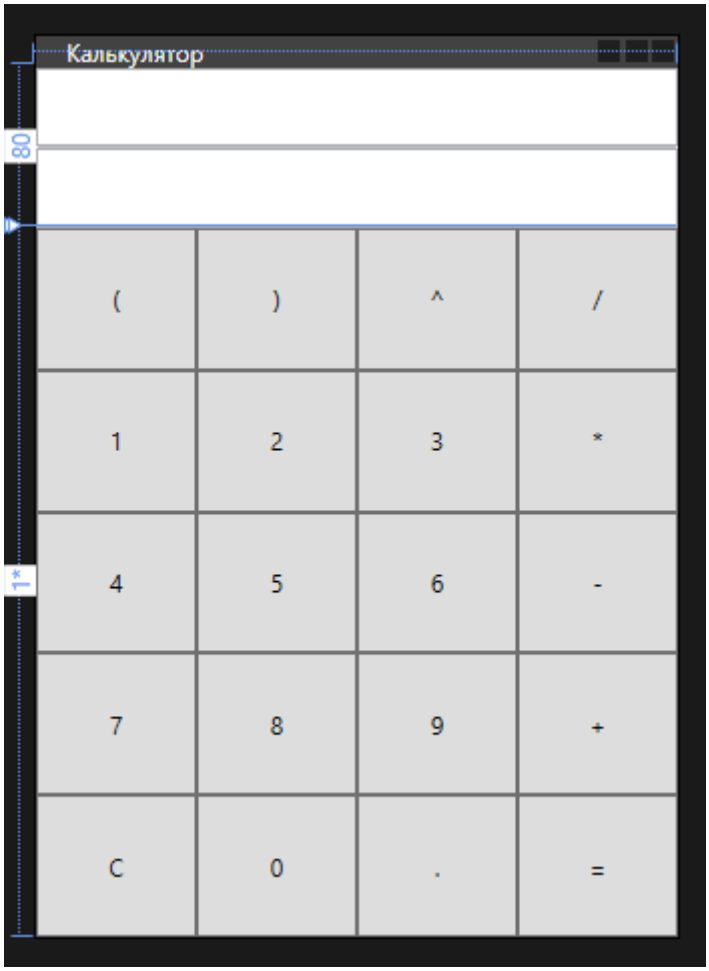
(Рис. 14)

Проектирование интерфейса WPF

В файле MainWindow.xaml создал интерфейс калькулятора.

После настройки XAML собрал проект — ошибок нет

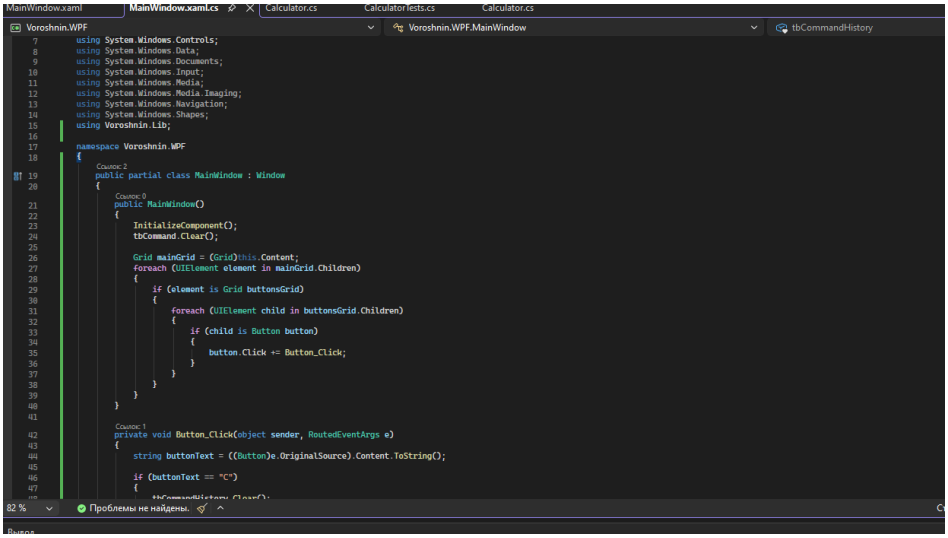
Вид калькулятора (Рис. 15)



(Рис. 15)

Обработка нажатий кнопок

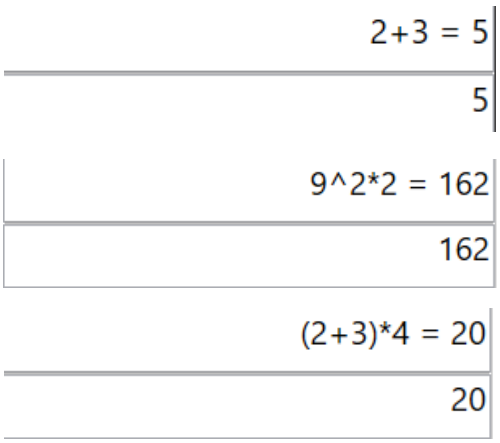
В файле MainWindow.xaml.cs создал обработчик событий для всех кнопок. В конструкторе через цикл нашёл все кнопки в Grid и подписал их на событие Click. Метод Button_Click обрабатывает три сценария: кнопка «C» очищает поля, кнопка «=» вычисляет выражение через Calculator.Run (с обработкой ошибок в блоке try/catch), остальные кнопки добавляют символ в поле ввода. После написания кода собрал проект — ошибок нет (Рис. 16)



(Рис. 16)

Запуск и финальное тестирование

Провел тестирование калькулятора (Рис. 17)

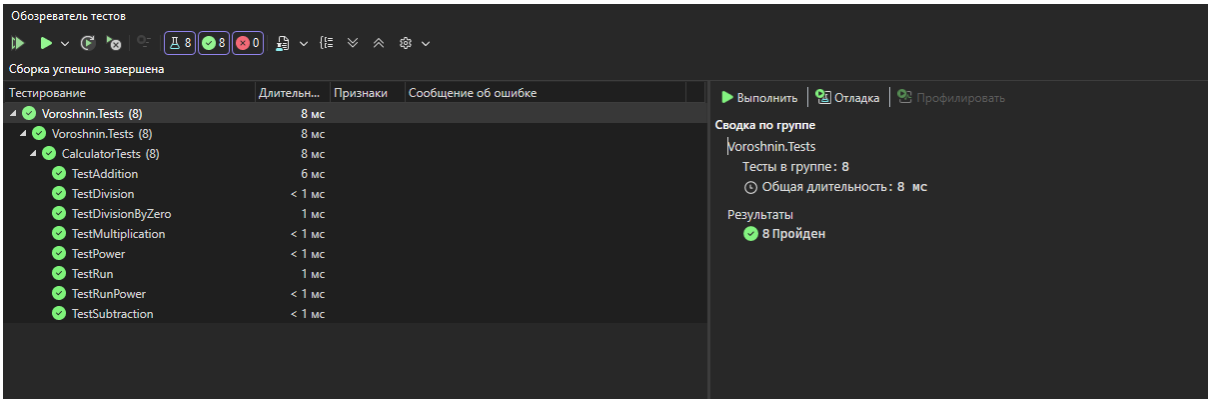


10/0

Ошибка: Деление на ноль

(Рис. 17)

Провел финальное тестирование (Рис. 18)



(Рис. 18)

Вывод

В ходе работы я разработал приложение-калькулятор. Начал с создания библиотеки классов, где написал методы для арифметических операций и разбора выражений. Затем добавил модульные тесты, чтобы проверить, что каждый метод работает правильно — все тесты прошли успешно. После этого сделал интерфейс на WPF: разместил кнопки, поля для ввода и истории, настроил обработку нажатий. В итоге приложение собирает без ошибок, тесты работают, калькулятор считает выражения, учитывает скобки и приоритеты операций, а при ошибках (например, деление на ноль) показывает понятное сообщение.